

EXPERIMENT NO: 4

Roll No.	09
Name	Mohish Padave
Class	D20B
Subject	Blockchain Technology Lab [NITPC71]
Lab Outcome	CO3: To learn Ethereum and develop/deploy smart contracts using Solidity and related tools/frameworks
Date of Performance / Submission	
Signature & Grades	

EXPERIMENT NO 4: STUDY OF SOLIDITY PROGRAM STRUCTURE, BASIC CONSTRUCTS COMPILATION AND DEPLOYMENT ENVIRONMENT

Aim: Hands on Solidity Programming Assignments for creating Smart Contracts

Theory:

1. Primitive Data Types, Variables, Functions – pure, view

In Solidity, primitive data types form the foundation of smart contract development. Commonly used types include:

- **uint / int:** unsigned and signed integers of different sizes (e.g., uint256, int128).
- **bool:** represents logical values (true or false).
- **address:** holds a 20-byte Ethereum account address, often used for storing user accounts or contract addresses.
- **bytes / string:** store binary data or textual data.

Variables in Solidity can be **state variables** (stored on the blockchain permanently), **local variables** (temporary, created during function execution), or **global variables** (special predefined variables such as msg.sender, msg.value, and block.timestamp).

Functions allow execution of contract logic. Special types of functions include:

pure: cannot read or modify blockchain state; they work only with inputs and internal computations.

view: can read state variables but cannot alter them. This classification helps optimize gas usage and enforces function integrity.

2. Inputs and Outputs to Functions

Functions in Solidity can accept input arguments and return one or more output values. Inputs enable users or other contracts to pass data into the contract, while outputs make it possible to return results after computation. For example, a function can accept an amount in Ether and return whether the transfer was successful. Solidity also allows named return variables, which improve readability and debugging.

3. Visibility, Modifiers and Constructors

Function Visibility defines who can access a function:

- **public**: available both inside and outside the contract.
- **private**: only accessible within the same contract.
- **internal**: accessible within the contract and its child contracts.
- **external**: can be called only by external accounts or other contracts.

Modifiers are reusable code blocks that change the behavior of functions. They are often used for access control, such as restricting sensitive functions to the contract owner (`onlyOwner`).

Constructors are special functions executed only once during contract deployment. They initialize important values, such as setting the deploying account as the owner of the contract.

3. Control Flow: if-else, loops

Control flow in Solidity is similar to traditional programming languages:

if-else allows conditional decision-making in contract logic, e.g., checking if a balance is sufficient before transferring funds.

Loops (`for`, `while`, `do-while`) enable repeated execution of code. For example, iterating through an array of users. However, loops must be used carefully, as excessive iterations increase gas consumption, potentially making the contract expensive to execute.

4. Data Structures: Arrays, Mappings, Structs, Enums

Arrays: Can be fixed or dynamic and are used to store ordered lists of elements.
Example: an array of addresses for registered users.

Mappings: Key-value pairs that allow quick lookups. Example: `mapping(address => uint)` for storing balances. Unlike arrays, mappings do not support iteration.

Structs: Allow grouping of related properties into a single data type, such as creating a `struct Player {string name; uint score;}`.

Enums: Used to define a set of predefined constants, making code more readable.
Example: `enum Status { Pending, Active, Closed }`.

5. Data Locations

Solidity uses three primary data locations for storing variables:

storage: Data stored permanently on the blockchain. Examples: state variables.

memory: Temporary data storage that exists only while a function is executing. Used for local variables and function inputs.

calldata: A non-modifiable and non-persistent location used for external function parameters. It is gas-efficient compared to memory. Understanding data locations is essential, as they directly impact gas costs and performance.

6. Transactions: Ether and Wei, Gas and Gas Price, Sending Transactions

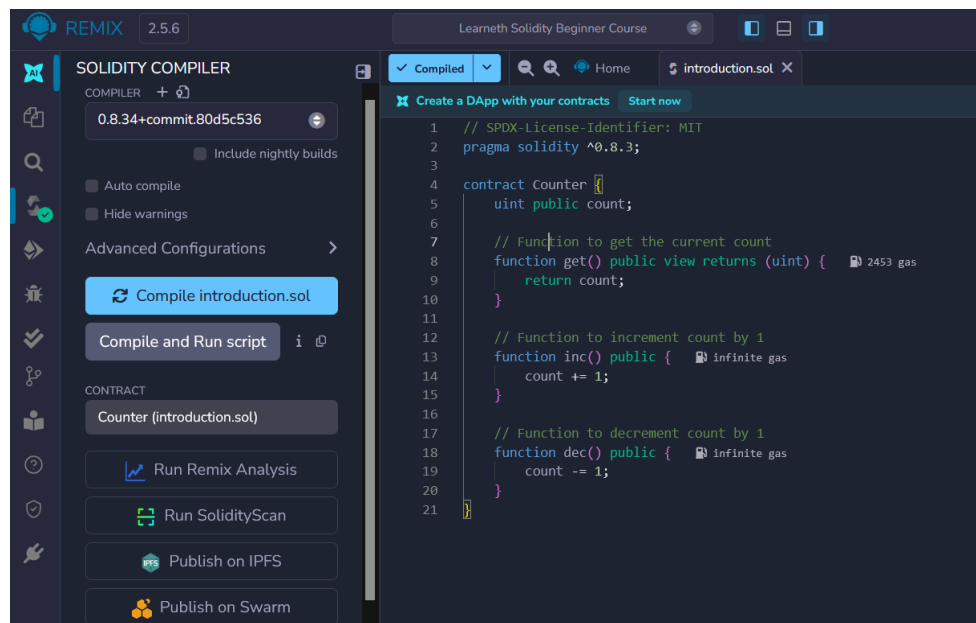
Ether and Wei: Ether is the main currency in Ethereum. All values are measured in Wei, the smallest unit (1 Ether = 10^{18} Wei). This ensures high precision in financial transactions.

Gas and Gas Price: Every transaction consumes gas, which represents computational effort. The gas price determines how much Ether is paid per unit of gas. A higher gas price incentivizes miners to prioritize the transaction.

Sending Transactions: Transactions are used for transferring Ether or interacting with contracts. Functions like `transfer()` and `send()` are commonly used, while `call()` provides more flexibility. Each transaction requires gas, making efficiency in contract design very important.

Implementation:

Tutorial no. 1 – Compile the code



Tutorial no. 1 – Deploy the contract

The screenshot shows the Remix IDE interface. On the left, the 'DEPLOY & RUN TRANSACTIONS' sidebar is open. It shows the 'Environment' section with 'Remix VM' and 'Osaka' selected. Below that, 'Account 1' is listed with a balance of 99,999 ETH. The 'Deploy' section shows the 'Counter' contract from 'introduction.sol' as 'Compiled'. The 'Value' is set to 0 wei, and the 'Gas limit' is set to 'auto'. A large blue 'Deploy' button is visible. Below the deployment section, there is a 'Deployed Contracts' section showing one contract.

The main editor displays the Solidity code for the 'Counter' contract:

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Counter {
5     uint public count;
6
7     // Function to get the current count
8     function get() public view returns (uint) { 2453 gas
9         return count;
10    }
11
12    // Function to increment count by 1
13    function inc() public { infinite gas
14        count += 1;
15    }
```

Below the code editor, the 'Explain contract' panel is open, showing instructions on how to interact with the contract and a list of accessible libraries (ethers.js).

Tutorial no. 1 – get

The screenshot shows the Remix IDE interface. On the left, the 'DEPLOY & RUN TRANSACTIONS' sidebar is open. It shows the 'Environment' section with 'Remix VM' and 'Osaka' selected. Below that, 'Account 1' is listed with a balance of 99,999 ETH. The 'Deploy' section shows the 'Counter' contract from 'introduction.sol' as 'Compiled'. The 'Value' is set to 0 wei, and the 'Gas limit' is set to 'auto'. A large blue 'Deploy' button is visible. Below the deployment section, there is a 'Deployed Contracts' section showing one contract.

The main editor displays the Solidity code for the 'Counter' contract:

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Counter {
5     uint public count;
6
7     // Function to get the current count
8     function get() public view returns (uint) { 2453 gas
9         return count;
10    }
11
12    // Function to increment count by 1
13    function inc() public { infinite gas
14        count += 1;
15    }
```

Below the code editor, the 'Explain contract' panel is open, showing instructions on how to interact with the contract and a list of accessible libraries (ethers.js).

Tutorial no. 1 – Increment

The screenshot shows the Remix IDE interface. On the left, the 'Functions' panel is open, displaying the 'inc' function under 'Low level interaction'. The 'Value' field is set to 0, and the 'Gas limit' is set to 'auto'. A 'Transact' button is visible. Below the panel, the 'Balance' is shown as 0 ETH. The main console on the right displays a 'revert' error message: 'The transaction has been reverted to the initial state. Note: The called function should be payable if you send value and the value you send should be less than your If the transaction failed for not having enough gas, try increasing the gas limit gently.. Error occurred: revert.' Below this, a transaction log entry is shown: '[vm] from: 0x5B3...eddC4 to: Counter.inc() 0xd91...39138 value: 0 wei data: 0x371...303c0 logs: 0 hash: 0xbd5...04b2b'. A 'Debug' button is next to the log entry.

Tutorial no. 1 – Decrement

The screenshot shows the Remix IDE interface. On the left, the 'Functions' panel is open, displaying the 'dec' function under 'Low level interaction'. The 'Value' field is set to 0, and the 'Gas limit' is set to 'auto'. A 'Transact' button is visible. Below the panel, the 'Balance' is shown as 0 ETH. The main console on the right displays a 'revert' error message: 'The transaction has been reverted to the initial state. Note: The called function should be payable if you send value and the value you send should be less than your If the transaction failed for not having enough gas, try increasing the gas limit gently.. Error occurred: revert.' Below this, two transaction log entries are shown: '[vm] from: 0x5B3...eddC4 to: Counter.inc() 0xd91...39138 value: 0 wei data: 0x371...303c0 logs: 0 hash: 0xbd5...04b2b' and '[vm] from: 0x5B3...eddC4 to: Counter.dec() 0xd91...39138 value: 0 wei data: 0xb3b...cfa82 logs: 0 hash: 0xf87...5fdc7'. Each log entry has a 'Debug' button next to it.

Tutorial no. 2

Tutorials list

Syllabus

2. Basic Syntax
2 / 19

variable when you declare it. In this case, `greet` is a `string`.

We also define the *visibility* of the variable, which specifies from where you can access it. In this case, it's a `public` variable that you can access from inside and outside the contract.

Don't worry if you didn't understand some concepts like *visibility*, *data types*, or *state variables*. We will look into them in the following sections.

To help you understand the code, we will link in all following sections to video tutorials from the [creator](#) of the Solidity by Example contracts.

Watch a video tutorial on [Basic Syntax](#).

★ Assignment

- Delete the HelloWorld contract and its content.
- Create a new contract named "MyContract".
- The contract should have a public state variable called "name" of the type string.
- Assign the value "Alice" to your new variable.

Check AnswerShow answer

Next

Well done! No errors.

Create a DApp with your contractsStart now

```
1 // SPDX-License-Identifier: MIT
2 // compiler version must be greater than or equal to 0.8.3 and less than 0.9.0
3 pragma solidity ^0.8.3;
4
5 contract MyContract {
6     string public name = "Alice";
7 }
```

✖ Explain contract

0 Listen on all transactions 🔍 Filter with transaction hash or

Tutorial no. 3

LEARNETH

Tutorials list

Syllabus

3. Primitive Data Types
3 / 19

You can learn more about these data types as well as *Fixed Point Numbers*, *Byte Arrays*, *Strings*, and more in the [Solidity documentation](#).

Later in the course, we will look at data structures like **Mappings**, **Arrays**, **Enums**, and **Structs**.

Watch a video tutorial on [Primitive Data Types](#).

★ Assignment

- Create a new variable `newAddr` that is a `public` `address` and give it a value that is not the same as the available variable `addr`.
- Create a `public` variable called `neg` that is a negative number, decide upon the type.
- Create a new variable, `newU` that has the smallest `uint` size type and the smallest `uint` value and is `public`.

Tip: Look at the other address in the contract or search the internet for an Ethereum address.

Check AnswerShow answer

Next

Well done! No errors.

Create a DApp with your contractsStart now

```
20 // negative numbers are allowed for int types.
21 // Like uint, different ranges are available from int8 to int256
22 */
23 int8 public i8 = -1;
24 int public i256 = 456;
25 int public i = -123; // int is same as int256
26
27 address public addr = 0xCA35b7d915458EF540aDe6068dFe2F44E8fa733c;
28
29 // Default values
30 // Unassigned variables have a default value
31 bool public defaultBool; // false
32 uint public defaultUint; // 0
33 int public defaultInt; // 0
34 address public defaultAddr; // 0x0000000000000000000000000000000000000000
35
36 address public newAddr = 0x0ADa31118866fF1aD0477F0C5D2e8eD35A36Eb5b;
37 int public neg = -10;
38 uint public newU = 0;
39 }
```

✖ Explain contract

0 Listen on all transactions 🔍 Filter with transaction hash or address

Welcome to Remix 2.5.7

Tutorial no. 4

LEARNETH

Tutorials list

Syllabus

4. Variables
4 / 19

retrieve information about the blockchain, particular addresses, contracts, and transactions.

In this example, we use `block.timestamp` (line 14) to get a Unix timestamp of when the current block was generated and `msg.sender` (line 15) to get the caller of the contract function's address.

A list of all Global Variables is available in the [Solidity documentation](#).

Watch video tutorials on [State Variables](#), [Local Variables](#), and [Global Variables](#).

★ Assignment

- Create a new public state variable called `blockNumber`.
- Inside the function `doSomething()`, assign the value of the current block number to the state variable `blockNumber`.

Tip: Look into the global variables section of the Solidity documentation to find out how to read the current block number.

Check Answer

Show answer

Next

Well done! No errors.

Compile

Home

QuickDApp

primitiveDataTypes.sol

variables

Create a DApp with your contracts

Start now

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Variables {
5     // State variables are stored on the blockchain.
6     string public text = "Hello";
7     uint public num = 123;
8     uint public blockNumber;
9
10    function doSomething() public { 22334 gas
11        // Local variables are not saved to the blockchain.
12        uint i = 456;
13
14        // Here are some global variables
15        uint timestamp = block.timestamp; // Current block timestamp
16        address sender = msg.sender; // address of the caller
17        blockNumber = block.number;
18    }
19 }
```

Explain contract

0 ☐ Listen on all transactions

Welcome to Remix 2.5.7

Tutorial no. 5

LEARNETH

Tutorials list

Syllabus

5.1 Functions - Reading and Writing to a State Variable
5 / 19

types and names. A common convention is to use an underscore as a prefix for the parameter name to distinguish them from state variables.

You can then set the visibility of a function and declare them `view` or `pure` as we do for the `get` function if they don't modify the state. Our `get` function also returns values, so we have to specify the return types. In this case, it's a `uint` since the state variable `num` that the function returns is a `uint`.

We will explore the particularities of Solidity functions in more detail in the following sections.

Watch a video tutorial on [Functions](#).

★ Assignment

- Create a public state variable called `b` that is of type `bool` and initialize it to `true`.
- Create a public function called `get_b` that returns the value of `b`.

Check Answer

Show answer

Next

Well done! No errors.

Compile

Home

QuickDApp

primitiveDataTypes.sol

variables.sol 3

Create a DApp with your contracts

Start now

```
3
4 contract SimpleStorage {
5     // State variable to store a number
6     uint public num;
7     bool public b = true;
8
9     // You need to send a transaction to write to a state variable.
10    function set(uint _num) public { 22536 gas
11        num = _num;
12    }
13
14    // You can read from a state variable without sending a transaction.
15    function get() public view returns (uint) { 2475 gas
16        return num;
17    }
18
19    function get_b() public view returns (bool) { 2539 gas
20        return b;
21    }
22 }
```

Explain contract

0 ☐ Listen on all transactions

Welcome to Remix 2.5.7

Tutorial no. 6

LEARNETH

Tutorials list

Syllabus

5.2 Functions - View and Pure

6 / 19

From the [Solidity documentation](#).

You can declare a pure function using the keyword `pure`. In this contract, `add` (line 13) is a pure function. This function takes the parameters `i` and `j`, and returns the sum of them. It neither reads nor modifies the state variable `x`.

In Solidity development, you need to optimise your code for saving computation cost (gas cost). Declaring functions view and pure can save gas cost and make the code more readable and easier to maintain. Pure functions don't have any side effects and will always return the same result if you pass the same arguments.

Watch a video tutorial on View and Pure Functions.

★ Assignment

Create a function called `addToX2` that takes the parameter `y` and updates the state variable `x` with the sum of the parameter and the state variable `x`.

Check Answer

Show answer

Next

Well done! No errors.

Compile

primitiveDataTypes.sol

variables.sol 3

readAndWrite.sol

Create a DApp with your contracts Start now

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract ViewAndPure {
5     uint public x = 1;
6
7     // Promise not to modify the state.
8     function addToX(uint y) public view returns (uint) { infinite gas
9         return x + y;
10    }
11
12    // Promise not to modify or read from the state.
13    function add(uint i, uint j) public pure returns (uint) { infinite gas
14        return i + j;
15    }
16
17    function addToX2(uint y) public { infinite gas
18        x = x + y;
19    }
20 }
```

✖ Explain contract

0 ☐ Listen on all transactions

Welcome to Remix 2.5.7

Tutorial no. 7

LEARNETH

Tutorials list

Syllabus

5.3 Functions - Modifiers and Constructors

7 / 19

contract initialization code. The constructor can have parameters and is especially useful when you don't know certain initialization values before the deployment of the contract.

You declare a constructor using the `constructor` keyword. The constructor in this contract (line 11) sets the initial value of the owner variable upon the creation of the contract.

Watch a video tutorial on Function Modifiers.

★ Assignment

- Create a new function, `increaseX` in the contract. The function should take an input parameter of type `uint` and increase the value of the variable `x` by the value of the input parameter.
- Make sure that `x` can only be increased.
- The body of the function `increaseX` should be empty.

Tip: Use modifiers.

Check Answer

Show answer

Next

Well done! No errors.

Compile

variables.sol 3

readAndWrite.sol

viewAndPure.sol

modifiers/

Create a DApp with your contracts Start now

```
44     _;
45     locked = false;
46 }
47 modifier greatthan0(uint y){
48     require(y > 0, "Input parameter not greater than 0");
49     _;
50 }
51
52 modifier actualincrease(uint y){
53     _;
54     x = x + y;
55 }
56
57 function decrement(uint i) public noReentrancy { infinite gas
58     x -= i;
59
60     if (i > 1) {
61         decrement(i - 1);
62     }
63 }
64
65 function increaseX(uint y) public onlyOwner greatthan0(y) actualincrease(y) {
66 }
```

✖ Explain contract

0 ☐ Listen on all transactions

Tutorial no. 8

LEARNETH

Tutorials list

Syllabus

5.4 Functions - Inputs and Outputs

8 / 19

contract functions.

"[Mappings] cannot be used as parameters or return parameters of contract functions that are publicly visible." From the [Solidity documentation](#).

Arrays can be used as parameters, as shown in the function `arrayInput` (line 71). Arrays can also be used as return parameters as shown in the function `arrayOutput` (line 76).

You have to be cautious with arrays of arbitrary size because of their gas consumption. While a function using very large arrays as inputs might fail when the gas costs are too high, a function using a smaller array might still be able to execute.

Watch a video tutorial on [Function Outputs](#).

★ Assignment

Create a new function called `returnTwo` that returns the values `-2` and `true` without using a return statement.

Check Answer

Show answer

Next

Well done! No errors.

Compile

AndWrite.sol

viewAndPure.sol

modifiersAndConstructors.sol

Create a DApp with your contracts

Start now

70

71

72

73

74

75

76

77

78

79

80

81

82

83

84

85

86

87

88

89

90

91

function arrayInput(uint[] memory _arr) public {} // infinite gas

// Can use array for input

uint[] public arr;

// Can use array for output

function arrayOutput() public view returns (uint[] memory) {} // infinite gas

return arr;

function returnTwo() // 472 gas

public

pure

returns (

int p,

bool q

)

{

p = -2;

q = true;

}

Explain contract

0

Listen on all transactions

Filter with transaction hash or address

Tutorial no. 9

LEARNETH

Tutorials list

Syllabus

6. Visibility

9 / 19

In this example, we have two contracts, the `Base` contract (line 4) and the `Child` contract (line 55) which inherits the functions and state variables from the `Base` contract.

When you uncomment the `testPrivateFunc` (lines 58-60) you get an error because the child contract doesn't have access to the private function `privateFunc` from the `Base` contract.

If you compile and deploy the two contracts, you will not be able to call the functions `privateFunc` and `internalFunc` directly. You will only be able to call them via `testPrivateFunc` and `testInternalFunc`.

Watch a video tutorial on [Visibility](#).

★ Assignment

Create a new function in the `Child` contract called `testInternalVar` that returns the values of all state variables from the `Base` contract that are possible to return.

Check Answer

Show answer

Next

Well done! No errors.

Compile

viewAndPure.sol

modifiersAndConstructors.sol

inputsAndOutputs.sol

Create a DApp with your contracts

Start now

50

51

52

53

54

55

56

57

58

59

60

61

62

63

64

65

66

67

68

69

70

string privateVar = "my private variable";

string public publicVar = "my public variable";

// State variables cannot be external so this code won't compile.

// string external externalVar = "my external variable";

}

contract Child is Base {

// Inherited contracts do not have access to private functions

// and state variables.

// function testPrivateFunc() public pure returns (string memory) {

// return privateFunc();

// }

// Internal function call be called inside child contracts.

function testInternalFunc() public pure override returns (string memory) {

return internalFunc();

}

function testInternalVar() public view returns (string memory, string memory) {

return (internalVar, publicVar);

}

}

Explain contract

0

Listen on all transactions

Filter with transaction hash or address

Tutorial no. 10

LEARNETH

Tutorials list

Syllabus

7.1 Control Flow - If/Else

10 / 19

else if

With the `else if` statement we can combine several conditions.

If the first condition (line 6) of the foo function is not met, but the condition of the `else if` statement (line 8) becomes true, the function returns `1`.

[Watch a video tutorial on the If/Else statement.](#)

★ Assignment

Create a new function called `evenCheck` in the `IfElse` contract:

- That takes in a `uint` as an argument.
- The function returns `true` if the argument is even, and `false` if the argument is odd.
- Use a ternary operator to return the result of the `evenCheck` function.

Tip: The modulo (%) operator produces the remainder of an integer division.

Check Answer

Show answer

Next

Well done! No errors.

Compile

modifiersAndConstructors.sol

inputsAndOutputs.sol

Create a DApp with your contracts

Start now

```
7      return 0;
8    } else if (x < 20) {
9      return 1;
10   } else {
11     return 2;
12   }
13 }
14
15 function ternary(uint _x) public pure returns (uint) {
16   // if (_x < 10) {
17   //   return 1;
18   // }
19   // return 2;
20
21   // shorthand way to write if / else statement
22   return _x < 10 ? 1 : 2;
23 }
24
25 function evenCheck(uint a) public pure returns (bool) {
26   return a % 2 == 0 ? true : false;
27 }
28 }
```

Explain contract

0 ☐ Listen on all transactions

Tutorial no. 11

LEARNETH

Tutorials list

Syllabus

7.2 Control Flow - Loops

11 / 19

break

The `continue` statement is used to skip the remaining code block and start the next iteration of the loop. In this contract, the `continue` statement (line 10) will prevent the second if statement (line 12) from being executed.

break

The `break` statement is used to exit a loop. In this contract, the break statement (line 14) will cause the for loop to be terminated after the sixth iteration.

[Watch a video tutorial on Loop statements.](#)

★ Assignment

1. Create a public `uint` state variable called count in the `Loop` contract.

2. At the end of the for loop, increment the count variable by 1.

3. Try to get the count variable to be equal to 9, but make sure you don't edit the `break` statement.

Check Answer

Show answer

Next

Well done! No errors.

Compile

AndConstructors.sol

inputsAndOutputs.sol

Create a DApp with your contracts

Start now

```
1  // SPDX-License-Identifier: MIT
2  pragma solidity ^0.8.3;
3
4  contract Loop {
5    uint public count;
6    function loop() public {
7      // for loop
8      for (uint i = 0; i < 10; i++) {
9        if (i == 5) {
10          // Skip to next iteration with continue
11          continue;
12        }
13        if (i == 5) {
14          // Exit loop with break
15          break;
16        }
17        count++;
18      }
19
20      // while loop
21      uint j;
22      while (j < 10) {
```

Explain contract

0 ☐ Listen on all transactions

Tutorial no. 12

LEARNETH

Tutorials list

Syllabus

8.1 Data Structures - Arrays

12 / 19

same, which means that the length of the array will stay the same. This will create a gap in our array. If the order of the array is not important, then we can move the last element of the array to the place of the deleted element (line 46), or use a mapping. A mapping might be a better choice if we plan to remove elements in our data structure.

Array length

Using the length member, we can read the number of elements that are stored in an array (line 35).

Watch a video tutorial on Arrays.

★ Assignment

1. Initialize a public fixed-sized array called `arr3` with the values 0, 1, 2. Make the size as small as possible.

2. Change the `getArr()` function to return the value of `arr3`.

Check Answer

Show answer

Next

Well done! No errors.

Compile

inputsAndOutputs.sol

visibility.sol

ifElse.sol

Create a DApp with your contracts

Start now

3

4 contract Array {

5 // Several ways to initialize an array

6 uint[] public arr;

7 uint[] public arr2 = [1, 2, 3];

8 uint[3] public arr3 = [0, 1, 2];

9 // Fixed sized array, all elements initialize to 0

10 uint[10] public myFixedSizeArr;

11

12 function get(uint i) public view returns (uint) { Infinite gas

13 return arr[i];

14 }

15

16 // Solidity can return the entire array.

17 // But this function should be avoided for

18 // arrays that can grow indefinitely in length.

19 function getArr() public view returns (uint[3] memory) { Infinite g

20 return arr3;

21 }

22

23 function push(uint i) public { 46828 gas

24 // Append to array

25 // This will increase the array length by 1.

26 }

27 }

28 }

29 }

30 }

31 }

32 }

33 }

34 }

35 }

36 }

37 }

38 }

39 }

40 }

41 }

42 }

43 }

44 }

45 }

46 }

47 }

48 }

49 }

50 }

51 }

52 }

53 }

54 }

55 }

56 }

57 }

58 }

59 }

60 }

61 }

62 }

63 }

64 }

65 }

66 }

67 }

68 }

69 }

70 }

71 }

72 }

73 }

74 }

75 }

76 }

77 }

78 }

79 }

80 }

81 }

82 }

83 }

84 }

85 }

86 }

87 }

88 }

89 }

90 }

91 }

92 }

93 }

94 }

95 }

96 }

97 }

98 }

99 }

100 }

101 }

102 }

103 }

104 }

105 }

106 }

107 }

108 }

109 }

110 }

111 }

112 }

113 }

114 }

115 }

116 }

117 }

118 }

119 }

120 }

121 }

122 }

123 }

124 }

125 }

126 }

127 }

128 }

129 }

130 }

131 }

132 }

133 }

134 }

135 }

136 }

137 }

138 }

139 }

140 }

141 }

142 }

143 }

144 }

145 }

146 }

147 }

148 }

149 }

150 }

151 }

152 }

153 }

154 }

155 }

156 }

157 }

158 }

159 }

160 }

161 }

162 }

163 }

164 }

165 }

166 }

167 }

168 }

169 }

170 }

171 }

172 }

173 }

174 }

175 }

176 }

177 }

178 }

179 }

180 }

181 }

182 }

183 }

184 }

185 }

186 }

187 }

188 }

189 }

190 }

191 }

192 }

193 }

194 }

195 }

196 }

197 }

198 }

199 }

200 }

201 }

202 }

203 }

204 }

205 }

206 }

207 }

208 }

209 }

210 }

211 }

212 }

213 }

214 }

215 }

216 }

217 }

218 }

219 }

220 }

221 }

222 }

223 }

224 }

225 }

226 }

227 }

228 }

229 }

230 }

231 }

232 }

233 }

234 }

235 }

236 }

237 }

238 }

239 }

240 }

241 }

242 }

243 }

244 }

245 }

246 }

247 }

248 }

249 }

250 }

251 }

252 }

253 }

254 }

255 }

256 }

257 }

258 }

259 }

260 }

261 }

262 }

263 }

264 }

265 }

266 }

267 }

268 }

269 }

270 }

271 }

272 }

273 }

274 }

275 }

276 }

277 }

278 }

279 }

280 }

281 }

282 }

283 }

284 }

285 }

286 }

287 }

288 }

289 }

290 }

291 }

292 }

293 }

294 }

295 }

296 }

297 }

298 }

299 }

300 }

301 }

302 }

303 }

304 }

305 }

306 }

307 }

308 }

309 }

310 }

311 }

312 }

313 }

314 }

315 }

316 }

317 }

318 }

319 }

320 }

321 }

322 }

323 }

324 }

325 }

326 }

327 }

328 }

329 }

330 }

331 }

332 }

333 }

334 }

335 }

336 }

337 }

338 }

339 }

340 }

341 }

342 }

343 }

344 }

345 }

346 }

347 }

348 }

349 }

350 }

351 }

352 }

353 }

354 }

355 }

356 }

357 }

358 }

359 }

360 }

361 }

362 }

363 }

364 }

365 }

366 }

367 }

368 }

369 }

370 }

371 }

372 }

373 }

374 }

375 }

376 }

377 }

378 }

379 }

380 }

381 }

382 }

383 }

384 }

385 }

386 }

387 }

388 }

389 }

390 }

391 }

392 }

393 }

394 }

395 }

396 }

397 }

398 }

399 }

400 }

401 }

402 }

403 }

404 }

405 }

406 }

407 }

408 }

409 }

410 }

411 }

412 }

413 }

414 }

415 }

416 }

417 }

418 }

419 }

420 }

421 }

422 }

423 }

424 }

425 }

426 }

427 }

428 }

429 }

430 }

431 }

432 }

433 }

434 }

435 }

436 }

437 }

438 }

439 }

440 }

441 }

442 }

443 }

444 }

445 }

446 }

447 }

448 }

449 }

450 }

451 }

452 }

453 }

454 }

455 }

456 }

457 }

458 }

459 }

460 }

461 }

462 }

463 }

464 }

465 }

466 }

467 }

468 }

469 }

470 }

471 }

472 }

473 }

474 }

475 }

476 }

477 }

478 }

479 }

480 }

481 }

482 }

483 }

484 }

485 }

486 }

487 }

488 }

489 }

490 }

491 }

492 }

493 }

494 }

495 }

496 }

497 }

498 }

499 }

500 }

501 }

502 }

503 }

504 }

505 }

506 }

507 }

508 }

509 }

510 }

511 }

512 }

513 }

514 }

515 }

516 }

517 }

518 }

519 }

520 }

521 }

522 }

523 }

524 }

525 }

526 }

527 }

528 }

529 }

530 }

531 }

532 }

533 }

534 }

535 }

536 }

537 }

538 }

539 }

540 }

541 }

542 }

543 }

544 }

545 }

546 }

547 }

548 }

549 }

550 }

551 }

552 }

553 }

554 }

555 }

556 }

557 }

558 }

559 }

560 }

561 }

562 }

563 }

564 }

565 }

566 }

567 }

568 }

569 }

570 }

571 }

572 }

573 }

574 }

575 }

576 }

577 }

578 }

579 }

580 }

581 }

582 }

583 }

584 }

585 }

586 }

587 }

588 }

589 }

590 }

591 }

592 }

593 }

594 }

595 }

596 }

597 }

598 }

599 }

600 }

601 }

602 }

603 }

604 }

605 }

606 }

607 }

608 }

609 }

610 }

611 }

612 }

613 }

614 }

615 }

616 }

617 }

618 }

619 }

620 }

621 }

622 }

623 }

624 }

625 }

626 }

627 }

628 }

629 }

630 }

631 }

632 }

633 }

634 }

635 }

636 }

637 }

638 }

639 }

640 }

641 }

642 }

643 }

644 }

645 }

646 }

647 }

648 }

649 }

650 }

651 }

652 }

653 }

654 }

655 }

656 }

657 }

658 }

659 }

660 }

661 }

662 }

663 }

664 }

665 }

666 }

667 }

668 }

669 }

670 }

671 }

672 }

673 }

674 }

675 }

676 }

677 }

678 }

679 }

680 }

681 }

682 }

683 }

684 }

685 }

686 }

687 }

688 }

689 }

690 }

691 }

692 }

693 }

694 }

695 }

696 }

697 }

698 }

699 }

700 }

701 }

702 }

703 }

704 }

705 }

706 }

707 }

708 }

709 }

710 }

711 }

712 }

713 }

714 }

715 }

716 }

717 }

718 }

719 }

720 }

721 }

722 }

723 }

724 }

725 }

726 }

727 }

728 }

729 }

730 }

731 }

732 }

733 }

734 }

735 }

736 }

737 }

738 }

739 }

740 }

741 }

742 }

743 }

744 }

745 }

746 }

747 }

748 }

749 }

750 }

751 }

752 }

753 }

754 }

755 }

756 }

757 }

758 }

759 }

760 }

761 }

762 }

763 }

764 }

765 }

766 }

767 }

768 }

769 }

770 }

771 }

772 }

773 }

774 }

775 }

776 }

777 }

778 }

779 }

780 }

781 }

782 }

783 }

784 }

785 }

786 }

787 }

788 }

789 }

790 }

791 }

792 }

793 }

794 }

795 }

796 }

797 }

798 }

799 }

800 }

801 }

802 }

803 }

804 }

805 }

806 }

807 }

808 }

809 }

810 }

811 }

812 }

813 }

814 }

815 }

816 }

817 }

818 }

819 }

820 }

821 }

822 }

823 }

824 }

825 }

826 }

827 }

828 }

829 }

830 }

831 }

832 }

833 }

834 }

835 }

836 }

837 }

838 }

839 }

840 }

841 }

842 }

843 }

844 }

845 }

846 }

847 }

848 }

849 }

850 }

851 }

852 }

853 }

854 }

855 }

856 }

857 }

858 }

859 }

860 }

861 }

862 }

863 }

864 }

865 }

866 }

867 }

868 }

869 }

870 }

871 }

872 }

873 }

874 }

875 }

876 }

877 }

878 }

879 }

880 }

881 }

882 }

883 }

884 }

885 }

886 }

887 }

888 }

889 }

890 }

891 }

892 }

893 }

894 }

895 }

896 }

897 }

898 }

899 }

900 }

901 }

902 }

903 }

904 }

905 }

906 }

907 }

908 }

909 }

910 }

911 }

912 }

913 }

914 }

915 }

916 }

917 }

918 }

919 }

920 }

921 }

922 }

923 }

924 }

925 }

926 }

927 }

928 }

929 }

930 }

931 }

932 }

933 }

934 }

935 }

936 }

937 }

938 }

939 }

940 }

941 }

942 }

943 }

944 }

945 }

946 }

947 }

948 }

949 }

950 }

951 }

952 }

953 }

954 }

955 }

956 }

957 }

958 }

959 }

960 }

961 }

962 }

963 }

964 }

965 }

966 }

967 }

968 }

969 }

970 }

971 }

972 }

973 }

974 }

975 }

976 }

977 }

978 }

979 }

980 }

981 }

982 }

983 }

984 }

985 }

986 }

987 }

988 }

989 }

990 }

991 }

992 }

993 }

994 }

995 }

996 }

997 }

998 }

999 }

1000 }

1001 }

1002 }

1003 }

1004 }

1005 }

1006 }

1007 }

1008 }

1009 }

1010 }

1011 }

1012 }

1013 }

1014 }

1015 }

1016 }

1017 }

1018 }

1019 }

1020 }

1021 }

1022 }

1023 }

1024 }

1025 }

1026 }

1027 }

1028 }

1029 }

1030 }

1031 }

1032 }

1033 }

1034 }

1035 }

1036 }

1037 }

1038 }

1039 }

1040 }

1041 }

1042 }

1043 }

1044 }

1045 }

1046 }

1047 }

1048 }

1049 }

1050 }

1051 }

1052 }

1053 }

1054 }

1055 }

1056 }

1057 }

1058 }

1059 }

1060 }

1061 }

1062 }

1063 }

1064 }

1065 }

1066 }

1067 }

1068 }

1069 }

1070 }

1071 }

1072 }

1073 }

1074 }

1075 }

1076 }

1077 }

1078 }

1079 }

1080 }

1081 }

1082 }

1083 }

1084 }

1085 }

1086 }

1087 }

1088 }

1089 }

1090 }

1091 }

1092 }

1093 }

1094 }

1095 }

1096 }

1097 }

1098 }

1099 }

1100 }

1101 }

1102 }

1103 }

1104 }

1105 }

1106 }

1107 }

1108 }

1109 }

1110 }

1111 }

1112 }

1113 }

1114 }

1115 }

1116 }

1117 }

1118 }

1119 }

1120 }

1121 }

1122 }

1123 }

1124 }

1125 }

1126 }

1127 }

1128 }

1129 }

1130 }

1131 }

1132 }

1133 }

1134 }

1135 }

1136 }

1137 }

1138 }

1139 }

1140 }

1141 }

1142 }

1143 }

1144 }

1145 }

1146 }

1147 }

1148 }

1149 }

1150 }

1151 }

1152 }

1153 }

1154 }

1155 }

1156 }

1157 }

1158 }

1159 }

1160 }

1161 }

1162 }

1163 }

1164 }

1165 }

1166 }

1167 }

1168 }

1169 }

1170 }

1171 }

1172 }

1173 }

1174 }

1175 }

1176 }

1177 }

1178 }

1179 }

1180 }

1181 }

1182 }

1183 }

1184 }

1185 }

1186 }

1187 }

1188 }

1189 }

1190 }

1191 }

1192 }

1193 }

1194 }

1195 }

1196 }

1197 }

1198 }

1199 }

1200 }

1201 }

1202 }

1203 }

1204 }

1205 }

1206 }

1207 }

1208 }

1209 }

1210 }

1211 }

1212 }

1213 }

1214 }

1215 }

1216 }

1217 }

1218 }

1219 }

1220 }

1221 }

1222 }

1223 }

1224 }

1225 }

1226 }

1227 }

1228 }

1229 }

1230 }

1231 }

1232 }

1233 }

1234 }

1235 }

1236 }

1237 }

1238 }

1239 }

1240 }

1241 }

1242 }

1243 }

1244 }

1245 }

1246 }

1247 }

1248 }

1249 }

1250 }

1251 }

1252 }

1253 }

1254 }

1255 }

1256 }

1257 }

1258 }

1259 }

1260 }

1261 }

1262 }

1263 }

1264 }

1265 }

1266 }

1267 }

1268 }

1269 }

1270 }

1271 }

1272 }

1273 }

1274 }

1275 }

1276 }

1277 }

1278 }

1279 }

1280 }

1281 }

1282 }

1283 }

1284 }

1285 }

1286 }

1287 }

1288 }

1289 }

1290 }

1291 }

1292 }

1293 }

1294 }

1295 }

1296 }

1297 }

1298 }

1299 }

1300 }

1301 }

1302 }

1303 }

1304 }

1305 }

1306 }

1307 }

1308 }

1309 }

1310 }

1311 }

1312 }

1313 }

1314 }

1315 }

1316 }

1317 }

1318 }

1319 }

1320 }

1321 }

1322 }

1323 }

1324 }

1325 }

1326 }

1327 }

1328 }

1329 }

1330 }

1331 }

1332 }

1333 }

1334 }

1335 }

1336 }

1337 }

1338 }

1339 }

1340 }

1341 }

1342 }

1343 }

1344 }

1345 }

1346 }

1347 }

1348 }

1349 }

1350 }

1351 }

1352 }

1353 }

1354 }

1355 }

1356 }

1357 }

1358 }

1359 }

1360 }

1361 }

1362 }

1363 }

1364 }

1365 }

1366 }

1367 }

1368 }

1369 }

1370 }

1371 }

1372 }

1373 }

1374 }

1375 }

1376 }

1377 }

1378 }

1379 }

1380 }

1381 }

1382 }

1383 }

1384 }

1385 }

1386 }

1387 }

1388 }

1389 }

1390 }

1391 }

1392 }

1393 }

1394 }

1395 }

1396 }

1397 }

1398 }

1399 }

1400 }

1401 }

1402 }

1403 }

1404 }

1405 }

1406 }

1407 }

1408 }

1409 }

1410 }

1411 }

1412 }

1413 }

1414 }

1415 }

1416 }

1417 }

1418 }

1419 }

1420 }

1421 }

1422 }

1423 }

1424 }

1425 }

1426 }

1427 }

1428 }

1429 }

1430 }

1431 }

1432 }

1433 }

1434 }

1435 }

1436 }

Tutorial no. 14

LEARNETH

Tutorials list

Syllabus

8.3 Data Structures - Structs

14 / 19

Initialize and update a struct: We initialize an empty struct first and then update its member by assigning it a new value (line 23).

Accessing structs

To access a member of a struct we can use the dot operator (line 33).

Updating structs

To update a struct's member we also use the dot operator and assign it a new value (lines 39 and 45).

Watch a video tutorial on Structs.

★ Assignment

Create a function `remove` that takes a `uint` as a parameter and deletes a struct member with the given index in the `todos` mapping.

Check Answer

Show answer

Next

Well done! No errors.

Create a DApp with your contracts

Start now

29

30

31

32

33

34

35

36

37

38

39

40

41

42

43

44

45

46

47

48

49

50

51

// solidity automatically created a getter for "todos" so

// you don't actually need this function.

function get(uint _index) public view returns (string memory text, bool completed) {

Todo storage todo = todos[_index];

return (todo.text, todo.completed);

}

// update text

function update(uint _index, string memory _text) public {

Todo storage todo = todos[_index];

todo.text = _text;

}

// update completed

function toggleCompleted(uint _index) public {

Todo storage todo = todos[_index];

todo.completed = !todo.completed;

}

function remove(uint _index) public {

delete todos[_index];

}

✖ Explain contract

0

Listen on all transactions

Filter with transaction hash or address

Tutorial no. 15

LEARNETH

Tutorials list

Syllabus

8.4 Data Structures - Enums

15 / 19

Updating an enum value

We can update the enum value of a variable by assigning it the `uint` representing the enum member (line 30). Shipped would be 1 in this example. Another way to update the value is using the dot operator by providing the name of the enum and its member (line 35).

Removing an enum value

We can use the delete operator to delete the enum value of the variable, which means as for arrays and mappings, to set the default value to 0.

Watch a video tutorial on Enums.

★ Assignment

1. Define an enum type called `Size` with the members `S`, `M`, and `L`.

2. Initialize the variable `sizes` of the enum type `Size`.

3. Create a getter function `getSize()` that returns the value of the variable `sizes`.

Check Answer

Show answer

Next

Well done! No errors.

Create a DApp with your contracts

Start now

12

13

14

15

16

17

18

19

20

21

22

23

24

25

26

27

28

29

30

31

32

33

34

35

36

37

38

enum Size {

S,

M,

L

}

// Default value is the first element listed in

// definition of the type, in this case "Pending"

Status public status;

Size public size;

// Returns uint

// Pending - 0

// Shipped - 1

// Accepted - 2

// Rejected - 3

// Canceled - 4

function get() public view returns (Status) {

return status;

}

function getSize() public view returns (Size) {

return size;

}

Tutorial no. 16

LEARNETH

Tutorials list

Syllabus

9. Data Locations

16 / 19

As we said in the beginning, when creating contracts we have to be mindful of gas costs. Therefore, we need to use data locations that require the lowest amount of gas possible.

★ Assignment

1. Change the value of the `myStruct` member `foo`, inside the `function f`, to 4.
2. Create a new struct `myMemStruct2` with the data location `memory` inside the `function f` and assign it the value of `myMemStruct`. Change the value of the `myMemStruct2` member `foo` to 1.
3. Create a new struct `myMemStruct3` with the data location `memory` inside the `function f` and assign it the value of `myStruct`. Change the value of the `myMemStruct3` member `foo` to 3.
4. Let the function `f` return `myStruct`, `myMemStruct2`, and `myMemStruct3`.

Tip: Make sure to create the correct return types for the function `f`.

Check Answer

Show answer

Next

Well done! No errors.

Create a DApp with your contracts

Start now

```
2 pragma solidity ^0.8.3;
3
4 contract DataLocations {
5     uint[] public arr;
6     mapping(uint => address) map;
7     struct MyStruct {
8         uint foo;
9     }
10    mapping(uint => MyStruct) myStructs;
11
12    function f() public returns (MyStruct memory, MyStruct memory, MyStruct memory) {
13        // call f with state variables
14        _f(arr, map, myStructs[1]);
15        // get a struct from a mapping
16        MyStruct storage myStruct = myStructs[1];
17        myStruct.foo = 4;
18        // create a struct in memory
19        MyStruct memory myMemStruct = MyStruct(0);
20        MyStruct memory myMemStruct2 = myMemStruct;
21        myMemStruct2.foo = 1;
22
23        MyStruct memory myMemStruct3 = myStruct;
24        myMemStruct3.foo = 3;
25        return (myStruct, myMemStruct2, myMemStruct3);
26    }
27
28    function _f(
```

Tutorial no. 17

LEARNETH

Tutorials list

Syllabus

10.1 Transactions - Ether and Wei

17 / 19

without a suffix are treated as `wei` (line 7).

`gwei`

One `gwei` (giga-wei) is equal to 1,000,000,000 (10^9) `wei`.

`ether`

One `ether` is equal to 1,000,000,000,000,000,000 (10^{18}) `wei` (line 11).

Watch a video tutorial on Ether and Wei.

★ Assignment

1. Create a `public uint` called `oneGwei` and set it to 1 `gwei`.
2. Create a `public bool` called `isOneGwei` and set it to the result of a comparison operation between 1 `gwei` and 10^9 .

Tip: Look at how this is written for `gwei` and `ether` in the contract.

Check Answer

Show answer

Next

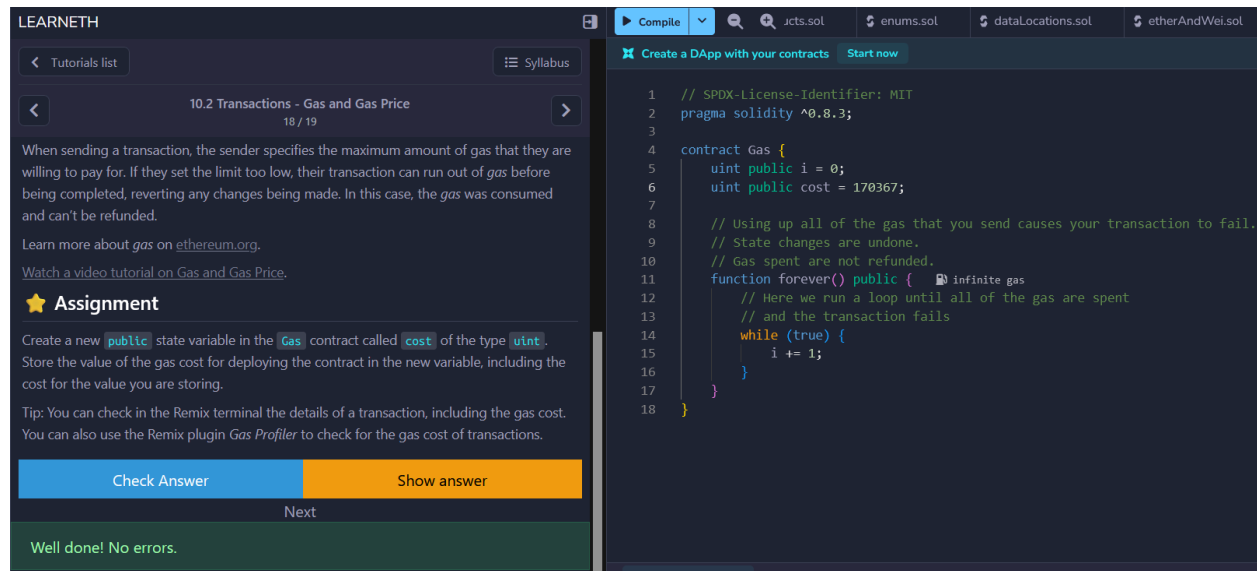
Well done! No errors.

Create a DApp with your contracts

Start now

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract EtherUnits {
5     uint public oneWei = 1 wei;
6     // 1 wei is equal to 1
7     bool public isOneWei = 1 wei == 1;
8
9     uint public oneEther = 1 ether;
10    // 1 ether is equal to 10^18 wei
11    bool public isOneEther = 1 ether == 1e18;
12
13    uint public oneGwei = 1 gwei;
14    bool public isOneGwei = 1 gwei == 1e9;
15 }
```

Tutorial no. 18



LEARNETH

Tutorials list | Syllabus

10.2 Transactions - Gas and Gas Price
18 / 19

When sending a transaction, the sender specifies the maximum amount of gas that they are willing to pay for. If they set the limit too low, their transaction can run out of gas before being completed, reverting any changes being made. In this case, the gas was consumed and can't be refunded.

Learn more about gas on ethereum.org.

Watch a video tutorial on Gas and Gas Price.

★ **Assignment**

Create a new `public` state variable in the `Gas` contract called `cost` of the type `uint`. Store the value of the gas cost for deploying the contract in the new variable, including the cost for the value you are storing.

Tip: You can check in the Remix terminal the details of a transaction, including the gas cost. You can also use the Remix plugin *Gas Profiler* to check for the gas cost of transactions.

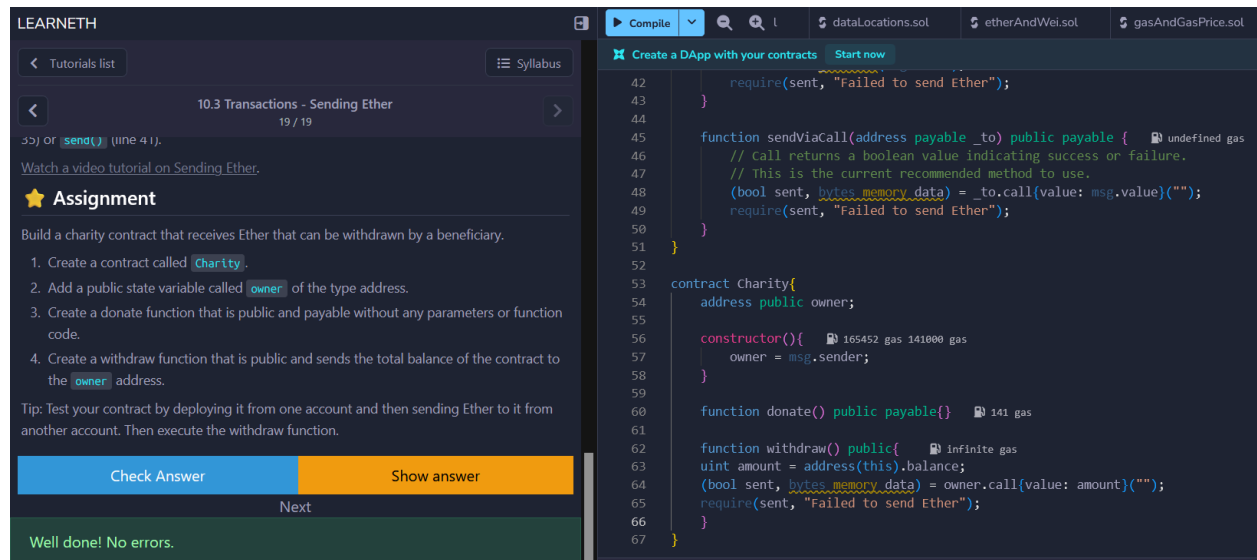
Check Answer | Show answer

Next

Well done! No errors.

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Gas {
5     uint public i = 0;
6     uint public cost = 170367;
7
8     // Using up all of the gas that you send causes your transaction to fail.
9     // State changes are undone.
10    // Gas spent are not refunded.
11    function forever() public { infinite gas
12        // Here we run a loop until all of the gas are spent
13        // and the transaction fails
14        while (true) {
15            i += 1;
16        }
17    }
18 }
```

Tutorial no. 19



LEARNETH

Tutorials list | Syllabus

10.3 Transactions - Sending Ether
19 / 19

or `send()` (line 41).

Watch a video tutorial on Sending Ether.

★ **Assignment**

Build a charity contract that receives Ether that can be withdrawn by a beneficiary.

1. Create a contract called `Charity`.
2. Add a public state variable called `owner` of the type `address`.
3. Create a donate function that is public and payable without any parameters or function code.
4. Create a withdraw function that is public and sends the total balance of the contract to the `owner` address.

Tip: Test your contract by deploying it from one account and then sending Ether to it from another account. Then execute the withdraw function.

Check Answer | Show answer

Next

Well done! No errors.

```
42     require(sent, "Failed to send Ether");
43 }
44
45 function sendViaCall(address payable to) public payable { undefined gas
46     // Call returns a boolean value indicating success or failure.
47     // This is the current recommended method to use.
48     (bool sent, bytes memory data) = to.call(value: msg.value)("");
49     require(sent, "Failed to send Ether");
50 }
51
52
53 contract Charity{
54     address public owner;
55
56     constructor(){ 165452 gas 141000 gas
57         owner = msg.sender;
58     }
59
60     function donate() public payable{ 141 gas
61
62     function withdraw() public{ infinite gas
63         uint amount = address(this).balance;
64         (bool sent, bytes memory data) = owner.call(value: amount)("");
65         require(sent, "Failed to send Ether");
66     }
67 }
```

Conclusion:

Through this experiment, the fundamentals of Solidity programming were explored by completing practical assignments in the Remix IDE. Concepts such as data types, variables, functions, visibility, modifiers, constructors, control flow, data structures, and transactions were implemented and understood. The hands-on practice helped in designing, compiling, and deploying smart contracts on the Remix VM, thereby strengthening the understanding of blockchain concepts. This experiment provided a strong foundation for developing and managing smart contracts efficiently.